Motifs e Discords em Séries Temporais com Matrix Profile

CEFET/RJ

Eduardo Ogasawara

eogasawara@cefet-rj.br | https://eic.cefet-rj.br/~eogasawara

Por que motifs importam

Motifs revelam padrões recorrentes que ajudam a explicar a estrutura da série temporal — e aparecem em domínios críticos do mundo real.



Saúde

Padrões em ECG, EEG e sinais fisiológicos revelam estados clínicos recorrentes.



Clima e Sensores

Ciclos climáticos e falhas em sensores aparecem como motifs repetidos.



Finanças

Padrões de preço e volume recorrentes podem antecipar comportamentos de mercado.

> 💡 A ausência de um motif esperado também pode sinalizar comportamento anômalo.

Motif discovery vs anomaly detection

🔍 Motif Discovery

- Procura repetição: subsequências similares que aparecem mais de uma vez.
- Resultado: conjunto de padrões recorrentes com suporte ≥ σ.
- Exemplo: batimento cardíaco normal que se repete no ECG.

⚠️ Anomaly Detection (Discords)

- Procura raridade ou desvio: subsequências incomuns ou isoladas.
- Resultado: subsequências sem vizinhos próximos (alto valor em M).
- Exemplo: arritmia que não se encaixa em nenhum padrão recorrente.

> Os dois problemas se conectam: o que não é motif pode ser discord.

Motif: definição, ocorrência e formalização

- Um motif é uma subsequência recorrente em uma série temporal.
- A subsequência não precisa ser conhecida previamente — é descoberta ao varrer a série.
- Extraímos de X todas as janelas deslizantes W de tamanho p.
- As ocorrências de q são as janelas suficientemente similares a q (distância ≤ ε ou similaridade ≥ limiar).
- q é motif quando o número de ocorrências atinge o suporte mínimo σ.

p = word size; ε = limiar de distância; σ = suporte mínimo; W = conjunto de janelas

Formalização

$$occurs(q, X) = \{w_i \mid w_i \in W \ \wedge \ dist(w_i, q) \leq \varepsilon\}$$

$$motif(q, X) \Leftrightarrow |occurs(q, X)| \geq \sigma$$

> A **exclusion zone** impede contar janelas vizinhas sobrepostas como ocorrências distintas: |i − j| > ez, com ez = p/2.

ez = exclusion zone; |i − j| = distância temporal entre janelas

Taxonomia de motif discovery

Dimensões da taxonomia

Representação

Como a série é reduzida ou codificada antes da busca. Exemplos: PAA (agregação), SAX (simbólica), z-score (normalização).

Similaridade

Qual função mede proximidade entre subsequências. Exemplos: Euclidiana (lock-step), DTW (elástica), Hamming (simbólica), MINDIST (lower bound SAX).

Método de busca

Como os candidatos são gerados e verificados. Exemplos: brute-force (exato), index-based (PAA/SAX), random projection (probabilístico), Matrix Profile (estrutural).

A descoberta depende de indexação, função de similaridade e método de busca.

Métodos de descoberta: visão geral

- Comparar todas as subsequências diretamente é caro: **O(n²) pares**.
- Indexação reduz dimensionalidade, discretiza valores ou organiza candidatos.
- O objetivo é acelerar a busca preservando padrões relevantes.
- A escolha do método depende do tamanho da série, da distância e do suporte desejado.

Brute-force

Compara tudo com tudo. Exato, mas $O(n^2 \cdot p)$. Referência de corretude.

Index-based

Reduz e discretiza via PAA/SAX. Agrupa candidatos simbolicamente.

Random Projection

Usa amostragem e colisões para identificar candidatos prováveis.

Matrix Profile

Estrutura todas as distâncias nearest-neighbor. Revela motifs e discords em uma única passagem.

Normalização: escala e métodos

- Duas subsequências podem ter a mesma forma em escalas diferentes.
- Sem normalização, amplitude e deslocamento dominam a distância.
- A normalização torna a comparação focada na forma do padrão.
- Z-score é o padrão em motif discovery: produz média zero e variância um.

$$z(x_t) = \frac{x_t - \mu}{\sigma}$$

μ = média da janela; σ = desvio padrão da janela

Z-Score

Produz média zero e variância um. Padrão em motif discovery e Matrix Profile.

Min-Max

Escala valores para [0, 1]. Útil quando a amplitude relativa importa.

Normalizações adaptativas

Lidam melhor com não estacionaridade local. Aplicadas janela a janela em vez de globalmente.

PAA: intuição e formalização

- Reduz a série por médias em segmentos de tamanho igual.
- O sinal fica mais compacto e menos sensível ao ruído local.
- É uma ponte natural para métodos simbólicos como SAX.

$$x'_j = \frac{n}{n'} \sum_{i=\frac{n}{n'}(j-1)+1}^{\frac{n}{n'}j} x_i$$

n = tamanho original; *n′* = número de segmentos; *p* = word size

ENTRADA: série X de tamanho n, número de segmentos n'
PRÉ-CONDIÇÃO: n divisível por n'; X normalizada (z-score)

PASSO 1 — Calcular tamanho do segmento: seg = n / n'
PASSO 2 — Para j de 1 até n':
  início = (j-1) * seg + 1
  fim   = j * seg
  x'[j] = média(X[início .. fim])
PASSO 3 — Retornar X' = [x'[1], x'[2], ..., x'[n']]

SAÍDA: série reduzida X' de tamanho n'
PÓS-CONDIÇÃO: cada x'[j] representa a média do j-ésimo segmento de X

SAX: representação simbólica

- Transforma valores numéricos em símbolos de um alfabeto de tamanho α.
- Breakpoints β dividem a distribuição normal em α faixas equiprováveis.
- Cada segmento PAA recebe o símbolo da faixa em que cai.
- Janelas de tamanho p viram palavras simbólicas de comprimento n'.

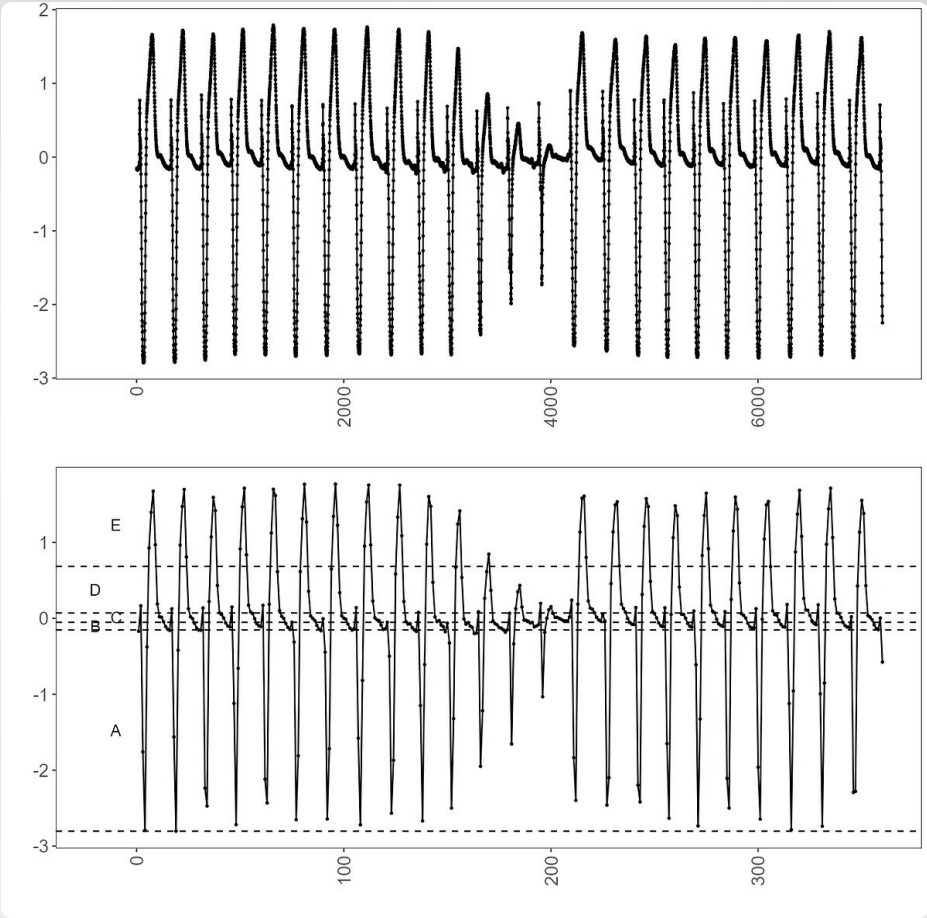
> **$x_t \in [\beta_{k-1}, \beta_k] \rightarrow$ símbolo a_k**

α = tamanho do alfabeto; β = breakpoints; n' = word size

```
ENTRADA: série X de tamanho n, word size n', alfabeto α
PRÉ-CONDIÇÃO: X normalizada (z-score); breakpoints β pré-calculados

PASSO 1 — Aplicar PAA: X' = PAA(X, n')
PASSO 2 — Para j de 1 até n':
   Para k de 1 até α:
      Se x'[j] ∈ [β[k-1], β[k]]:
         símbolo[j] = a_k
         parar busca
PASSO 3 — Palavra = concatenar(símbolo[1..n'])
PASSO 4 — Repetir para cada janela deslizante de X

SAÍDA: sequência de palavras simbólicas
PÓS-CONDIÇÃO: cada palavra representa uma janela de tamanho p de X
```



SAX: propriedades e distância MINDIST

- SAX preserva a estrutura de vizinhança: palavras próximas correspondem a subsequências próximas.
- MINDIST é um lower bound da distância Euclidiana entre subsequências originais.
- Permite descartar candidatos sem calcular a distância numérica completa.
- Quanto maior o alfabeto α, menor o erro de aproximação.

$$MINDIST(\hat{q}, \hat{q}') = \sqrt{\frac{p}{n'}} \sum_{i=1}^{n'} dist(\hat{q}_i, \hat{q}'_i)$$

p = tamanho da janela; n' = word size; dist = tabela de distâncias entre símbolos

Tabela de distâncias simbólicas (α = 4)

	a	b	c	d
a	0	0	0.67	1.34
b	0	0	0	0.67
c	0.67	0	0	0
d	1.34	0.67	0	0

Valores baseados nos breakpoints da distribuição normal padrão para α=4.

Similaridade: medidas e escolha

- Motif discovery depende de decidir quando duas subsequências são parecidas.
- A escolha da medida de distância muda quais padrões serão considerados motifs.
- Não há medida universal: a escolha depende do fenômeno e das distorções esperadas.
- Medidas lock-step comparam ponto a ponto com alinhamento rígido.
- Medidas elásticas permitem desalinhamento temporal e variações de ritmo.

Lock-step

Compara posições correspondentes diretamente. Simples e eficiente. Exemplos: Euclidiana, Hamming. Sensível a deslocamentos temporais.

Elástica

Permite alinhar pontos fora de ordem. Captura similaridade de forma mesmo com variações de ritmo. Exemplo clássico: DTW.

> "A escolha entre lock-step e elástica é uma das decisões mais impactantes no pipeline de motif discovery."

Distância Euclidiana

- Compara duas sequências de mesmo tamanho ponto a ponto.
- É simples, rápida e amplamente usada em motif discovery.
- Funciona bem quando as subsequências estão alinhadas no tempo.
- Sensível a diferenças de amplitude — requer normalização prévia.

$$EDist(q, q') = \sqrt{\sum_{i=1}^{p} (q_i - q'_i)^2}$$

p = tamanho das sequências; qi, q'i = valores nas posições correspondentes

Propriedades

Complexidade

O(p) por par de subsequências. Permite early-abandon para poda eficiente.

Lower bound

LB_Keogh e LB_Kim são lower bounds da Euclidiana usados para poda em buscas indexadas.

Limitação

Não tolera deslocamentos temporais. Duas sequências com a mesma forma mas defasadas terão distância alta.

Distância de Hamming

- Conta posições diferentes entre sequências simbólicas de mesmo tamanho.
- É natural para palavras SAX: mede quantas letras diferem.
- Mede quantas substituições seriam necessárias para igualar as sequências.
- Usada para agrupar candidatos a motif no pipeline index-based.

$$HDist(q, q') = \sum_{i=1}^{p} HItem(q_i, q'_i)$$

$$HItem(x, y) = \begin{cases} 0 & \text{se } x = y \\ 1 & \text{caso contrário} \end{cases}$$

p = word size; *qi*, *q'i* = símbolos nas posições correspondentes

Propriedades e uso

Complexidade

O(p) por par. Muito eficiente para palavras SAX curtas.

Agrupamento

Janelas com HDist ≤ h são candidatas ao mesmo motif. Controla o trade-off entre recall e precisão.

Limitação

Ignora a magnitude da diferença entre símbolos. MINDIST corrige isso usando a tabela de distâncias simbólicas.

DTW: intuição e motivação

- Dynamic Time Warping permite alinhar sequências que evoluem em ritmos diferentes.
- O método procura um caminho de menor custo em uma matriz de alinhamento.
- É útil quando a forma importa mais do que o alinhamento exato no tempo.
- Cada ponto de uma sequência pode ser alinhado a um ou mais pontos da outra.
- A restrição de Sakoe-Chiba (banda w) limita o desvio máximo de alinhamento.

Construir matriz de custo

Cada célula (i, j) recebe o custo de alinhar q[i] com q'[j].

Encontrar caminho ótimo

Programação dinâmica encontra o caminho de menor custo da célula (1,1) até (p,p).

Calcular distância

A distância DTW é a raiz do custo acumulado no caminho ótimo.

DTW é mais robusto que a Euclidiana para séries com variações de ritmo, mas tem custo $O(p^2)$ por par.

DTW: programação dinâmica

- A matriz acumula o menor custo para alinhar prefixos das duas sequências.
- Cada célula depende de três vizinhos já calculados.
- O custo final é a distância elástica entre as sequências.
- Restrição de Sakoe-Chiba (banda w) limita o desvio máximo de alinhamento.

$$dtw_{i,j} = custo(q_i, q'_j) + \min(dtw_{i-1,j},\ dtw_{i,j-1},\ dtw_{i-1,j-1})$$

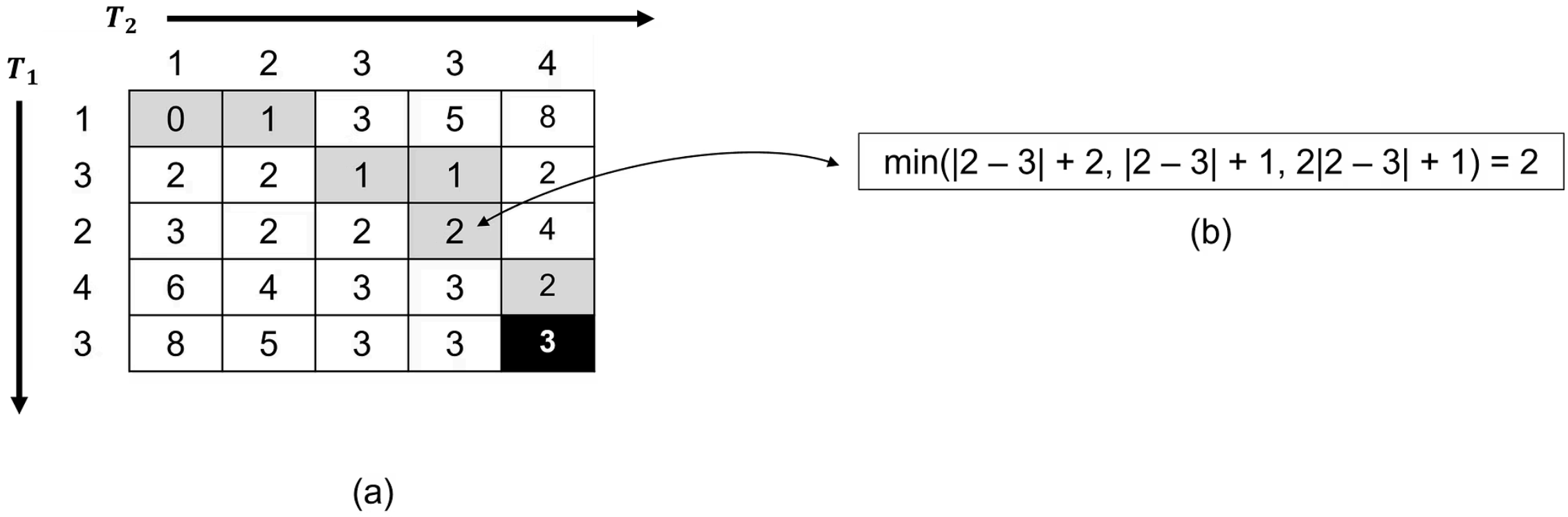
p = tamanho das sequências; w = largura da banda de restrição

```
ENTRADA: sequências q e q' de tamanho p, banda w
PRÉ-CONDIÇÃO: q e q' normalizadas; w ≥ 0

PASSO 1 — Inicializar matriz DTW[0..p][0..p] com ∞
        DTW[0][0] = 0
PASSO 2 — Para i de 1 até p:
   Para j de max(1, i-w) até min(p, i+w):
      custo = (q[i] - q'[j])²
      DTW[i][j] = custo +
         min(DTW[i-1][j], DTW[i][j-1], DTW[i-1][j-1])
PASSO 3 — Retornar sqrt(DTW[p][p])

SAÍDA: distância DTW entre q e q'
PÓS-CONDIÇÃO: valor ≥ 0; igual a zero se q = q'
```

Exemplo visual de DTW



DTW entre duas sequências pequenas, mostrando matriz de custo e cálculo por programação dinâmica.

Lower bounds e poda

Lower bounds

Estimam um limite inferior da distância real. Se o limite já supera ε, o par é descartado sem calcular a distância completa. Exemplos: LB_Keogh (DTW), LB_Yi, LB_Kim.

Early-abandon

Interrompe o cálculo ponto a ponto assim que o acumulado excede ε^2. Compatível com distância Euclidiana e DTW.

Poda por indexação

Índices simbólicos (SAX) ou estruturas de dados (iSAX, R-tree) eliminam regiões inteiras do espaço de busca antes de qualquer cálculo de distância.

> Combinar lower bounds com early-abandon pode reduzir o custo prático em ordens de magnitude.

Brute-force discovery: intuição

Brute-force discovery

- Extrai todas as subsequências de tamanho *p* da série.
- Compara cada subsequência com todas as demais (exceto exclusion zone).
- Conta ocorrências: janelas com distância abaixo de ε.
- É exato, mas complexidade $O(n^2)$ vezes o custo da distância.

p = word size; ε = limiar de distância; σ = suporte mínimo; ez = exclusion zone

```
ENTRADA: série X de tamanho n, tamanho p, limiar ε, suporte σ
PRÉ-CONDIÇÃO: X normalizada; ez = p/2 (exclusion zone padrão)

PASSO 1 — W = extrair todas as janelas de X de tamanho p
PASSO 2 — motifs = {}
PASSO 3 — Para cada w_i em W:
  ocorrências = 0
  Para cada w_j em W, |i - j| > ez:
    Se dist(w_i, w_j) ≤ ε:
      ocorrências += 1
  Se ocorrências ≥ σ:
    motifs.adicionar(w_i)
PASSO 4 — Retornar motifs
SAÍDA: conjunto de subsequências motif
PÓS-CONDIÇÃO: cada motif ocorre ≥ σ vezes com distância ≤ ε
```

Brute-force: algoritmo e complexidade

Brute-force: complexidade e otimizações

- Complexidade total: $O(n^2 \cdot p)$ para distância Euclidiana.
- Com DTW, o custo por par é $O(p^2)$, elevando para $O(n^2 \cdot p^2)$.
- Early-abandon reduz o custo prático sem alterar a exatidão.
- Normalização prévia é essencial para comparações significativas.

n = tamanho da série; *p* = tamanho da janela

ENTRADA: sequências w_i e w_j de tamanho p, limiar ε
PRÉ-CONDIÇÃO: w_i e w_j normalizadas

PASSO 1 — acumulado = 0
PASSO 2 — Para t de 1 até p:
  acumulado += $(w_i[t] - w_j[t])^2$
  Se acumulado > ε^2:
    Retornar ∞ (abandono antecipado)
PASSO 3 — Retornar sqrt(acumulado)

SAÍDA: distância Euclidiana ou ∞ se excede ε
PÓS-CONDIÇÃO: resultado ∞ descarta o par sem custo total

Index-based discovery: PAA/SAX

Teoria

- Evita comparar diretamente todas as subsequências numéricas.
- Usa PAA e SAX para representar janelas como palavras simbólicas.
- Janelas com a mesma palavra (ou palavras próximas) são candidatas a motif.
- Grupos com suporte suficiente são verificados numericamente.

p = word size; *α* = alfabeto; *σ* = suporte mínimo; *ε* = limiar de verificação

Pseudocódigo

ENTRADA: série X, tamanho p, alfabeto α, suporte σ, limiar ε
PRÉ-CONDIÇÃO: X normalizada (z-score)

PASSO 1 — W = extrair janelas deslizantes de X de tamanho p
PASSO 2 — Para cada w_i em W:
  X'_i = PAA(w_i, n')
  palavra_i = SAX(X'_i, α)
PASSO 3 — Agrupar janelas por palavra simbólica
PASSO 4 — Para cada grupo com |grupo| ≥ σ:
  Verificar numericamente: dist(w_i, w_j) ≤ ε
  Se confirmado: registrar como motif candidato
PASSO 5 — Retornar motifs confirmados

SAÍDA: conjunto de motifs com suporte ≥ σ
PÓS-CONDIÇÃO: falsos positivos eliminados na verificação numérica

Index-based: agrupamento e verificação

Index-based: agrupamento e verificação

- Janelas simbólicas iguais ou próximas (distância de Hamming ≤ h) são agrupadas.
- Grupos com suporte suficiente (|grupo| ≥ σ) são candidatos a motif.
- A verificação numérica elimina falsos positivos introduzidos pela discretização.
- O índice simbólico pode ser uma hash table ou trie sobre palavras SAX.

h = limiar de Hamming; *σ* = suporte mínimo; *ε* = limiar numérico de verificação

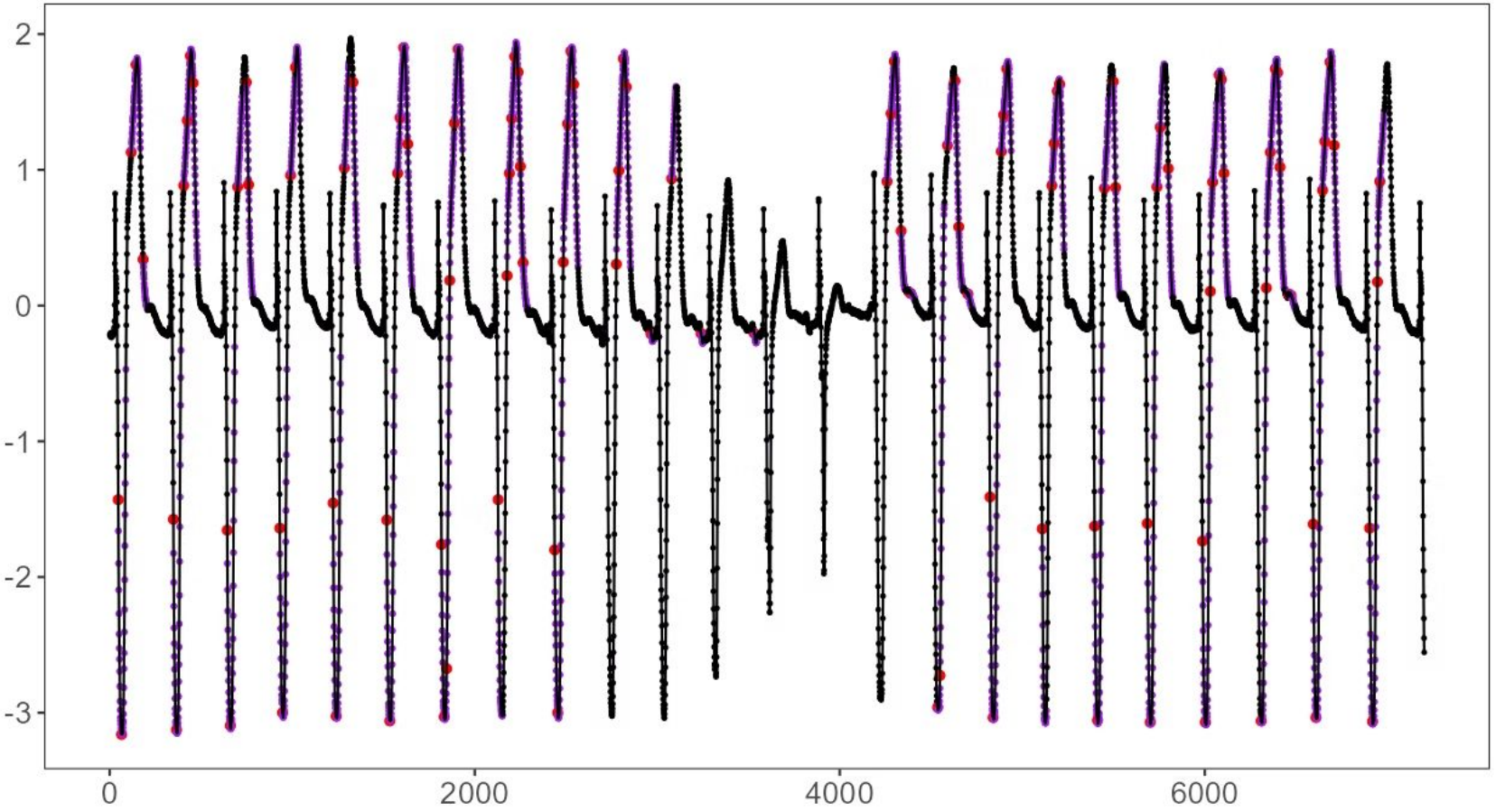
Agrupamento simbólico (pseudocódigo)

```
ENTRADA: palavras SAX {palavra_i}, suporte σ, Hamming h, limiar ε
PRÉ-CONDIÇÃO: palavras calculadas para todas as janelas

PASSO 1 — índice = hash_table()
PASSO 2 — Para cada (i, palavra_i):
   índice[palavra_i].adicionar(i)
PASSO 3 — candidatos = {}
PASSO 4 — Para cada (chave_a, grupo_a) em índice:
   Para cada (chave_b, grupo_b) em índice:
      Se HDist(chave_a, chave_b) ≤ h:
         candidatos.unir(grupo_a ∪ grupo_b)
PASSO 5 — Para cada grupo em candidatos com |grupo| ≥ σ:
   Verificar numericamente pares do grupo com dist ≤ ε

SAÍDA: motifs confirmados
PÓS-CONDIÇÃO: falsos positivos eliminados; falsos negativos possíveis
```

Motifs detectados com SAX



Index-Based Discovery encontra motifs em batimentos regulares no ECG.

Random projection: intuição e algoritmo

- Projetar reduz a dimensão considerada na comparação.
- Subsequências similares tendem a colidir nas mesmas projeções aleatórias.
- Uma matriz de colisões acumula evidência probabilística de similaridade.
- Motifs são selecionados entre candidatos com suporte alto na matriz.

*Parâmetros: **n'** = word size SAX; **r** = número de posições projetadas; **iter** = iterações; **σ** = suporte mínimo*

```
ENTRADA: série X, word size n', posições r, iterações iter, suporte σ
PRÉ-CONDIÇÃO: X normalizada; palavras SAX calculadas

PASSO 1 — Calcular palavras SAX para todas as janelas de X
PASSO 2 — Colisões = matriz n × n inicializada com zeros
PASSO 3 — Para t de 1 até iter:
    proj = escolher r posições aleatórias de [1..n']
    bucket = {}
    Para cada janela w_i:
        chave = projetar(palavra_i, proj)
        bucket[chave].adicionar(i)
    Para cada grupo em bucket:
        Para cada par (i, j) no grupo, |i-j| > ez:
            Colisões[i][j] += 1
PASSO 4 — Candidatos = pares com Colisões[i][j] ≥ σ
PASSO 5 — Verificar candidatos numericamente

SAÍDA: motifs confirmados por colisão e verificação
PÓS-CONDIÇÃO: pares com alta colisão têm alta
              probabilidade de ser motif
```

Random projection: análise probabilística

- A probabilidade de colisão de duas palavras similares aumenta com o número de iterações.
- Subsequências com distância de Hamming pequena colidem com alta probabilidade.
- Subsequências distantes raramente colidem — o método tem baixa taxa de falsos positivos.
- A verificação numérica final elimina os falsos positivos restantes.

$$P(\text{colisão}) = \left(\frac{\alpha - HDist}{\alpha}\right)^{r}$$

r = posições projetadas; *α* = tamanho do alfabeto; *HDist* = distância de Hamming

Interpretação da matriz de colisões

Alta colisão

Par (i, j) com Colisões[i][j] alto → candidato forte a motif. Verificar numericamente.

Baixa colisão

Par (i, j) com Colisões[i][j] baixo → provavelmente não é motif. Descartar sem verificação.

Suporte mínimo σ

Threshold sobre a matriz de colisões. Controla o trade-off entre recall e custo de verificação.

Early-abandon e lower bounds

- Early-abandon interrompe o cálculo quando o erro parcial já excede ε.
- Economiza tempo evitando completar comparações que não podem virar match.
- Lower bounds estimam um limite inferior da distância real sem calcular a distância completa.
- Se o lower bound já supera ε, o par é descartado sem custo adicional.
- LB_Keogh é o lower bound clássico para DTW: usa envelope de largura w.

$$LB_Keogh(q, q') \leq DTW(q, q')$$

w = largura da banda Sakoe-Chiba; ε = limiar de distância

LB_Keogh (pseudocódigo)

ENTRADA: sequência q, envelope (U, L) de q', limiar ε
PRÉ-CONDIÇÃO: U[t] = max(q'[t-w..t+w]); L[t] = min(q'[t-w..t+w])

PASSO 1 — lb = 0
PASSO 2 — Para t de 1 até p:
  Se q[t] > U[t]:
    lb += (q[t] - U[t])²
  Senão se q[t] < L[t]:
    lb += (q[t] - L[t])²
  Se lb > ε²:
    Retornar ∞ (poda antecipada)
PASSO 3 — Retornar sqrt(lb)

SAÍDA: lower bound de DTW(q, q') ou ∞ se excede ε
PÓS-CONDIÇÃO: LB_Keogh ≤ DTW real; nunca superestima

Matrix Profile: intuição e algoritmo

- Para cada subsequência, guarda a distância até sua subsequência mais parecida (nearest neighbor).
- O vetor **M** guarda a menor distância; o índice **I** guarda a posição do vizinho.
- Em self-join, uma **exclusion zone** evita matches triviais próximos da diagonal.
- Distâncias baixas → **motifs**; distâncias altas → **discords**.

$$M[i] = \min_{j,\ |i-j|>ez} z\text{-}norm\text{-}dist(w_i, w_j)$$

p = tamanho da janela; *ez* = exclusion zone (padrão p/2);
n = tamanho da série

ENTRADA: série X de tamanho n, tamanho de janela p
PRÉ-CONDIÇÃO: n > 2p; ez = floor(p/2)

PASSO 1 — W = extrair todas as janelas de X de tamanho p
PASSO 2 — M = vetor de tamanho (n-p+1) inicializado com ∞
I = vetor de tamanho (n-p+1) inicializado com -1
PASSO 3 — Para cada par (i, j), |i - j| > ez:
d = z-norm-dist(W[i], W[j])
Se d < M[i]: M[i] = d; I[i] = j
Se d < M[j]: M[j] = d; I[j] = i
PASSO 4 — Retornar (M, I)

SAÍDA: Matrix Profile M e índice I
PÓS-CONDIÇÃO: M[i] = distância ao vizinho mais próximo de W[i]
I[i] = posição desse vizinho

Matrix Profile: definição formal

- M é um vetor de tamanho (n - p + 1).
- M[i] = distância z-normalizada de W[i] ao seu nearest neighbor (excluindo exclusion zone).
- I[i] = posição do nearest neighbor de W[i].
- Em self-join, a exclusion zone evita matches triviais: |i - j| ≤ ez.
- Em AB-join, compara duas séries distintas X e Y.

$$M[i] = \min_{j:\,|i-j|>ez} \|z(W_i) - z(W_j)\|_2$$

z(·) = z-normalização; *ez* = floor(p/2); *p* = tamanho da janela

Propriedades

Completude

Contém informação suficiente para encontrar todos os motifs e discords de ordem k sem recalcular distâncias.

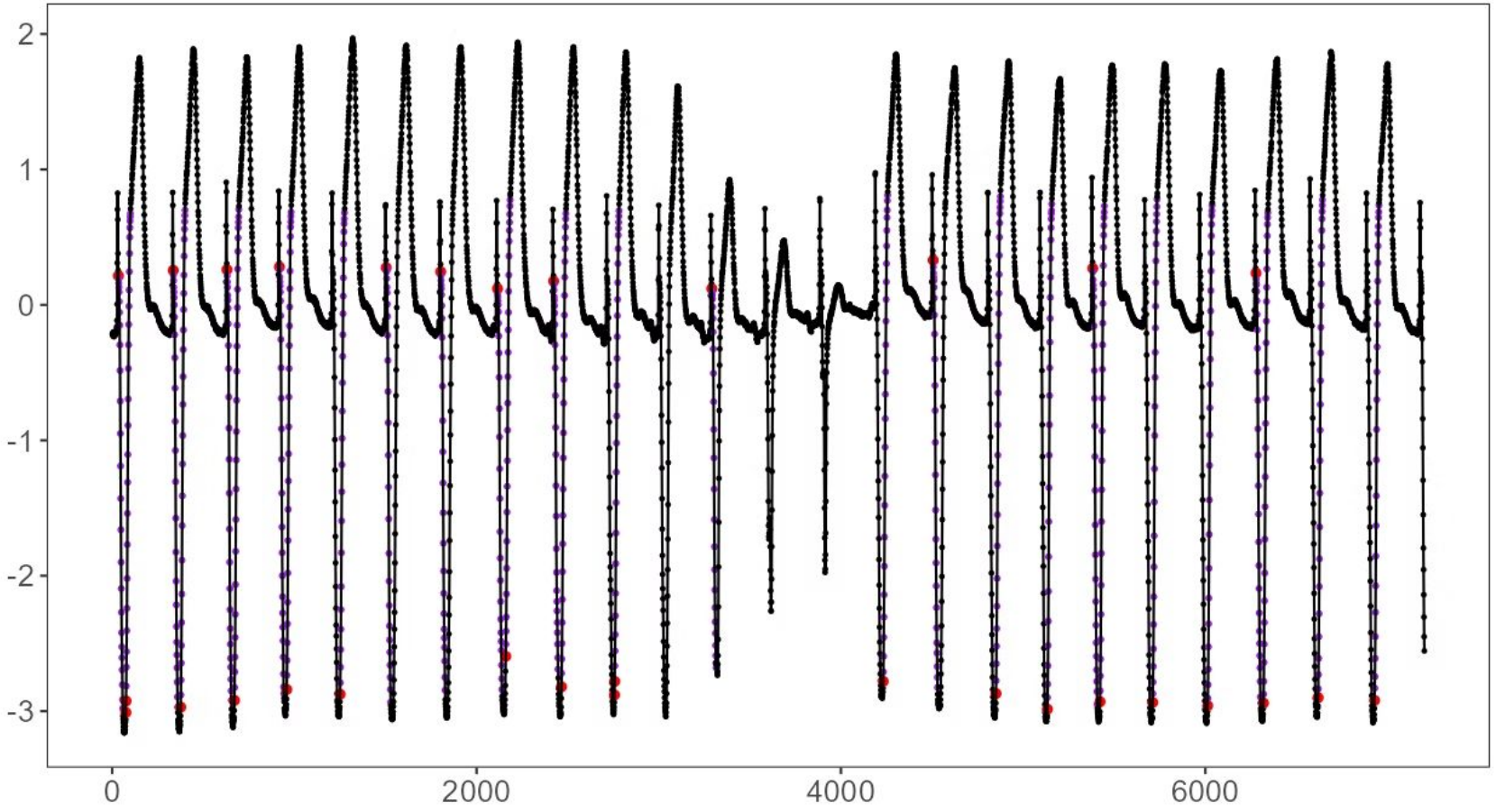
Incrementalidade

Pode ser atualizado à medida que novos dados chegam (SWAMP para streaming).

Generalidade

Funciona com qualquer medida de distância que admita z-normalização. Extensível para múltiplas dimensões (mSTAMP).

Motifs detectados com Matrix Profile



Matrix Profile com STAMP também encontra motifs em batimentos regulares no ECG.

Matrix Profile também revela discords

- O mesmo vetor **M** que revela motifs também aponta subsequências isoladas.
- Isso acontece porque **M** mede a distância ao vizinho mais próximo.
- Se até o vizinho mais próximo está longe, a subsequência é candidata a discord.
- Motifs e discords são leituras opostas do mesmo vetor **M**.

min(M) → Motif

Subsequência com vizinho mais próximo. Padrão recorrente.

max(M) → Discord

Subsequência mais isolada. Candidata a anomalia.

> "Uma única passagem pelo Matrix Profile revela tanto os padrões mais comuns quanto os mais raros da série."

- Não é necessário rodar dois algoritmos separados.
- A ordem de busca (motifs primeiro, discords depois) pode ser ajustada conforme o objetivo.
- Discords de ordem *k* são encontrados iterativamente, excluindo regiões já selecionadas.

Menores valores: motifs

- Um valor baixo em M[i] indica que existe uma subsequência muito parecida com W[i].
- O motif de ordem 1 é o par (i, j) com menor valor em M.
- Motifs de ordem k são encontrados iterativamente, excluindo regiões já selecionadas.
- Ocorrências adicionais podem ser buscadas a partir desses pares.

$$motif_1 = \arg\min_i M[i]$$

ez = exclusion zone; k = número de motifs desejados

ENTRADA: Matrix Profile M, índice I, exclusion zone ez, k motifs
PRÉ-CONDIÇÃO: M e I calculados; ez = floor(p/2)

PASSO 1 — M_temp = cópia de M
PASSO 2 — motifs = []
PASSO 3 — Para t de 1 até k:
  i* = argmin(M_temp)
  j* = I[i*]
  motifs.adicionar((i*, j*))
  Para pos em [i* - ez .. i* + ez]:
    M_temp[pos] = +∞ (excluir região)
  Para pos em [j* - ez .. j* + ez]:
    M_temp[pos] = +∞ (excluir região)
PASSO 4 — Retornar motifs

SAÍDA: lista de k pares de motifs em ordem crescente de distância
PÓS-CONDIÇÃO: cada par é o motif mais próximo na região restante

Maiores valores: discords

Discords por Matrix Profile

- Um valor alto em M indica que a subsequência não tem vizinhos próximos.
- O discord de ordem 1 é a subsequência com maior valor em M.
- Discords de ordem k são encontrados iterativamente, excluindo regiões já selecionadas.
- Essa leitura conecta motif discovery com detecção de anomalias intervalares.

$$discord_1 = \arg\max_i M[i]$$

ez = exclusion zone; *k* = número de discords desejados

ENTRADA: Matrix Profile M, índice I, exclusion zone ez, k discords
PRÉ-CONDIÇÃO: M e I calculados; ez = floor(p/2)

PASSO 1 — M_temp = cópia de M
PASSO 2 — discords = []
PASSO 3 — Para t de 1 até k:
  i* = argmax(M_temp)
  discords.adicionar(i*)
  Para j em [i* - ez .. i* + ez]:
    M_temp[j] = -∞ (excluir região)
  Para j tal que I[j] ∈ [i* - ez .. i* + ez]:
    recalcular M_temp[j] sem i*
PASSO 4 — Retornar discords

SAÍDA: lista de k posições de discords em ordem decrescente de isolamento
PÓS-CONDIÇÃO: cada discord é a subsequência mais isolada na região restante

Conexão entre os eventos

Motifs

Recorrência — subsequências que se repetem com distância ≤ ε. Representam estados ou comportamentos típicos da série.

Discords

Isolamento — subsequências sem vizinhos próximos (alto valor em M). Representam anomalias ou eventos raros.

Eventos intervalares

Ambos dependem de subsequência, distância e escala temporal. São a base para análise de eventos em séries temporais.

> Matrix Profile unifica a descoberta de motifs e discords em uma única estrutura.

Motifs multivariados

Podem ser tratados por redução de dimensionalidade, combinação de descobertas ou busca conjunta.

Redução de dimensionalidade

PCA, autoencoders ou projeções reduzem as dimensões antes da busca. Perde relações entre variáveis se não for cuidadoso.

Combinação de descobertas

Motifs são descobertos por dimensão e depois combinados por co-ocorrência temporal. Simples, mas ignora correlações cruzadas.

Busca conjunta (mSTAMP)

Estende Matrix Profile para múltiplas dimensões simultaneamente. Captura relações entre variáveis sem explodir o espaço de busca.

> Em muitos casos, motifs multivariados aparecem na interação entre variáveis — não em cada uma isoladamente.

Tamanho variável de motifs

- Muitos métodos assumem tamanho fixo p — mas o tamanho real do padrão pode ser desconhecido.
- Buscar em múltiplos valores de p é custoso: **$O(n^2 \cdot |P|)$** para um conjunto P de tamanhos.
- **PMP** (Pan Matrix Profile) calcula Matrix Profile para todos os tamanhos de uma vez.
- **VALMOD** usa busca hierárquica para encontrar motifs de tamanho variável eficientemente.

P = conjunto de tamanhos candidatos; $p$min, $p$max = faixa de busca

Estratégias

Definir faixa [p_min, p_max]

Estabelecer os limites mínimo e máximo dos tamanhos candidatos a investigar.

Calcular Matrix Profile para cada p

Usar PMP ou VALMOD para cobrir toda a faixa de tamanhos de forma eficiente.

Selecionar motifs ótimos

Escolher motifs com menor distância normalizada pelo tamanho p.

Ranking de motifs e ocorrências

Nem todo padrão recorrente é interessante. Motifs podem ser ranqueados por diferentes critérios para identificar os mais relevantes.

Suporte

Número de ocorrências do motif. Motifs com mais ocorrências são mais representativos da série.

Similaridade

Distância média entre as ocorrências. Motifs com menor distância interna são mais coesos.

Entropia / Raridade

Motifs muito frequentes podem ser triviais. Raridade relativa pode indicar padrões mais informativos.

Densidade

Distribuição temporal das ocorrências. Motifs concentrados em regiões específicas podem indicar regimes ou estados.

> Padrões quase constantes ou trivialmente similares devem ser descartados mesmo com suporte alto.

Big data e streaming

Desafios

- Séries longas tornam comparação e armazenamento desafiadores.
- Streaming exige atualização incremental do Matrix Profile.
- Memória limitada impede manter toda a série em RAM.
- Paralelismo e computação distribuída são necessários para escala.

Soluções

iSAX

Índice hierárquico para séries longas em disco.

SWAMP

Atualização incremental do Matrix Profile em streaming.

mSTAMP

Matrix Profile multivariado com suporte a paralelismo.

Disk-aware

Algoritmos que minimizam acesso a disco para séries que não cabem em memória.

Novas estruturas e métodos

STAMP, STOMP, SCRIMP, PMP e VALMOD

STAMP

Calcula Matrix Profile por pares aleatórios; suporta modo anytime.

STOMP

Explora localidade entre colunas adjacentes para reduzir custo; $O(n^2)$ com constante menor.

SCRIMP

Amostra diagonais da matriz de distâncias; anytime e paralelizável.

PMP

Calcula Matrix Profile para múltiplos tamanhos de janela simultaneamente.

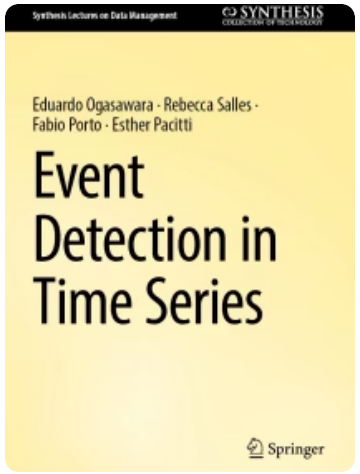
VALMOD

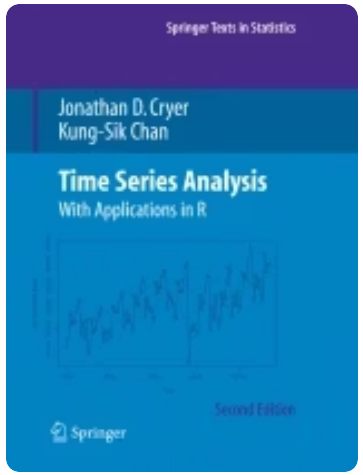
Busca motifs de tamanho variável sem fixar *p* a priori.

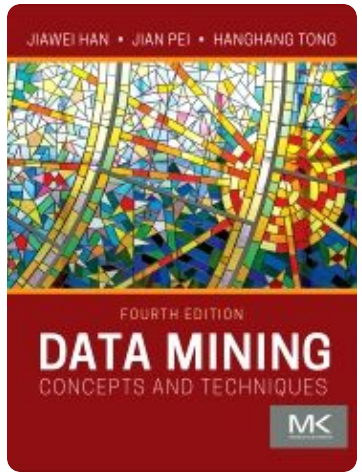
Comparação dos algoritmos

Algoritmo	Estratégia	Anytime?	Tam. variável?
STAMP	Pares aleatórios	Sim	Não
STOMP	Diagonais locais	Não	Não
SCRIMP	Diagonais amostradas	Sim	Não
PMP	Múltiplos *p*	Sim	Sim (via agregação)
VALMOD	Busca hierárquica	Não	Sim

Referências Bibliográficas







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4ª ed., 2022.